Marking Scheme

Module Code: COMP I8062

Module Title: Algorithms

Internal Examiner(s): Dr Michelle Graham

Session: SAMPLE

Year: 24-25

Instructions:

Answer QUESTION ONE (40 marks) and any other TWO questions (30 marks each).

Marking Scheme

Marking Scheme

You MUST attempt this question.

Question 1) Algorithm Design and Complexity

(a) Explain the following algorithm design paradigms. For each paradigm, illustrate your answer with a suitable example of an algorithm you have studied.

(i) Backtracking.
(ii) Recursive.
(iii) Two pointer.

(12 marks)

MARKS BREAKDOWN:

Explanation: 3 marks * 3
Example algorithm: 1 mark * 3

(b) Examine the algorithm found in **appendix A**. It has been designed to process bell-shaped data, i.e. data that ascends to a peak, then decreases.

(i) Assume the algorithm is run with the following array as its input:
data = [8, 15, 16, 21, 23, 30, 29, 28, 16, 14, 12, 8, 6].
What is the result of this algorithm? Explain how this result is reached.

(8 marks)

SAMPLE ANSWER:

Result: 30 (peak of the data)
Student should trace the progress of the algorithm through the data.

MARKS BREAKDOWN:

Result: 1 mark
Identification of result as peak: 1 mark
Tracing progress: 6 marks

Marking Scheme

(ii) What is the paradigm being used for the algorithm's design? Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

SAMPLE ANSWER:

Brute force.

Space: O(1) (should state no data structures created & no recursive calls)

Time: O(n) (should point to need to potentially iterate through entire array)

MARKS BREAKDOWN:

Naming paradigm:	1 mark
Complexity:	0.5 mark * 2
Complexity explanation:	1 mark * 2
Code justification:	1 mark * 2

(iii) Write and explain the **code** for an alternative (but still functional) solution to this problem that employs recursion instead of iteration. Indicate and explain the base case and recursive case in your implementation.

(14 marks)

SAMPLE ANSWER:

Pseudocode for the following (should include validation):

Marking Scheme

```java
public static int findPeak(int[] data) {
    int start = 0;
    int end = data.length - 1;

    return findPeak(data, start, end);
}

private static int findPeak(int [] data, int start, int end){
    if(start >= end){
        return data[start];
    }

    int index = start + (end - start) / 2;
    if (data[index] < data[index + 1]) {
        start = index + 1;
    } else {
        end = index;
    }

    return findPeak(data, start, end);
}
```

MARKS BREAKDOWN:

Pseudocode:	5 marks
Explanation:	5 marks
Base case:	2 marks
Recursive case:	2 marks

Choose any TWO questions from questions 2, 3 and 4

Question 2) Sorting Algorithms

(a) What is meant by stability in sorting? Give an example where stability is required.

(5 marks)

SAMPLE ANSWER:

Mark

"Stability in Sorting refers to a property of sorting algorithms where elements with equal keys (or values) maintain their relative order after sorting. In other words, if two items are equal in terms of the sorting key, a stable sort will ensure that they appear in the same order in the sorted output as they did in the original input."

Example requiring stability: Multi-level/multi-attribute sorting

MARKS BREAKDOWN:

Explanation: 3 marks

Example: 2 marks

(b) Compare and contrast **Bubble sort** vs **Quick sort** under the following headings:

(i) Time complexity

(ii) Space complexity

(iii) Stability

(15 marks)

SAMPLE ANSWER:

See notes on Bubble sort and Quick sort

MARKING SCHEME:

Discussion under heading: 5 * 3

(c) Consider the following array: **[22, 18, 5, 8, 10]**

Apply the **Selection Sort** algorithm to sort this array into ascending order. Show the array after each completed pass of the sorting process until it is fully sorted.

(10 marks)

Pass 1:

Index	0	1	2	3	4

Marking Scheme

Value	5	18	22	8	10

Pass 2:

Index	0	1	2	3	4
Value	5	8	22	18	10

Pass 3:

Index	0	1	2	3	4
Value	5	8	10	18	22

Pass 4:

Index	0	1	2	3	4
Value	5	8	10	18	22

MARKS BREAKDOWN:

Passes: 3 marks
Showing position: 2 marks
Correct action: 3 marks
Noting algorithm doesn't end even though array is sorted: 2 marks

Question 3) Recursive Algorithms

(a) The code shown in **appendix B** is a recursive algorithm to calculate a specified Fibonacci number.

(i) Explain the three essential components required for a recursive function to work correctly. For each component, describe why it is necessary.

(6 marks)

SAMPLE ANSWER:

- Base case – stops recursion process, triggers stack calls to start being popped back off
- Recursive case (action to take on each stage) – computes result/carries out logic
- Progress towards base case (way to reduce the data towards the base case) – won't stop if the data doesn't reduce

MARKS BREAKDOWN:

Name & explain components: 1.5 mark * 3

Marking Scheme

Necessity: 0.5 marks * 3

(ii) Identify and explain how each of the three essential components of recursion is implemented in the algorithm shown in **appendix B**.

(6 marks)

SAMPLE ANSWER:

Base case 1: if n < 0

Base case 2: if n <= 1

Base case 3: if n is in map

Recursive case: calculating the previous two Fibonacci numbers (i.e. F(n) = F(n-2) + F(n-1))

Reduction: Decreasing n by 2 and by 1

MARKS BREAKDOWN:

Identifying: 1 mark * 3

Explaining: 1 mark * 3

(iii) Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

SAMPLE ANSWER:

Time: $O(2^n)$ (every iteration triggers 2 new calls but unlike merge sort and quick sort, the data set is not significantly reducing)

Space: O(n) (n recursive calls on stack, plus n for hash map storage, but 2n reduces to n in Big O)

MARKS BREAKDOWN:

Complexity: 1 mark * 2

Explanation: 1 mark * 2

Justification: 1 mark * 2

Marking Scheme

(b) This implementation of the Fibonacci algorithm employs a form of optimisation.

(i) What optimisation technique has been used here? What is the purpose of this form of optimisation? Illustrate your answer with reference to specific lines of code.

(4 marks)

SAMPLE ANSWER:

Memoization is an optimization technique used in recursive algorithms to store the results of expensive function calls and reuse them when the same inputs occur again. Memoization stores computed results in a cache (usually a dictionary or an array) so that each unique function call only needs to be computed once.

MARKS BREAKDOWN:

Explanation: 1 mark
How it optimises: 2 marks (should mention saving on recursive calls AND processing time)
Reference to code: 1 mark

(ii) How does the inclusion of this technique impact the space and time complexities incurred by this algorithm? Explain your answer.

(8 marks)

MARKS BREAKDOWN:

Cost of cache (Map): 1 mark
Storing result: 3 marks
Checking result: 2 marks
Complexity impact: 2 marks (Worst case doesn't change, average case improves.)

Marking Scheme

Question 4) Selecting Algorithms

(a) Your team has been tasked with developing a ticketing system for raising issues internally, with tickets modelled as a queue of Ticket objects stored as an array. The Tickets are to be stored in order of priority, i.e. the Ticket with the highest priority level should always appear first in the queue. You have been assigned to develop a feature that adds a new ticket to the system. The application is envisioned to work on a device with standard power and memory availability, and the application places an emphasis on response time.

(i) Outline two approaches to *array insertion* suitable for use in this context. Ensure that you have adapted them to maintain the sorted nature of the data **without** the need to sort the data after insertion.

(14 marks)

SAMPLE ANSWER:

1) Shift insertion
2) Resize insertion
3) Valid alternative approach

Must find appropriate position into which it should insert – this can be done as part of process or prior. Either is acceptable if overall complexity is not impacted.

Should note that shift insertion will lose data unless there is empty space in array.

MARKS BREAKDOWN:

Explanation: 7 marks * 2

(ii) Which of the two approaches you have discussed would you recommend for use in the scenario above, and why? Justify your answer with evidence based on the manner of operation of the algorithms, and any relevant details from the given scenario.

Marking Scheme

SAMPLE ANSWER:

Any reasonable recommendation is acceptable if it has been appropriately justified and contradictory information has been considered.

MARKS BREAKDOWN:

Recommendation:	1 mark
Rationale:	3 marks
Relevant details:	2 marks

(b) Following a caching error, it was discovered that tickets have been added to the ticket queue out of order, rendering the entire ticket queue unsorted. You have been assigned the task of resolving this issue. Rather than sorting the array and continuing as normal, your teammate suggests creating a new array and adding all tickets to that array in sorted order.

(i) Write a short note comparing the performance of your teammate's suggested approach to that of sorting the existing array.

(6 marks)

SAMPLE ANSWER:

Short outline of process

Performance of sort and then insert is at best O(n log n) (depending on sort algorithm)

Performance of insert-in-place is at worst O(n)

MARKS BREAKDOWN:

Outline:	1 marks
Performance comparisons:	5 marks

(ii) Which of the approaches you have discussed would you recommend for use in the scenario above, and why? Justify your answer with evidence based on the manner of operation of the algorithms, and any relevant details from the given scenario.

Marking Scheme

(4 marks)

SAMPLE ANSWER:

Any reasonable recommendation is acceptable if it has been appropriately justified and contradictory information has been considered.

MARKS BREAKDOWN:

Recommendation:	1 mark
Rationale:	2 marks
Relevant details:	1 marks

Marking Scheme

APPENDIX A:

```
public static int altFindPeak(int [] data){
    for (int i = 0; i < data.length-1; i++) {
        if(data[i] > data[i+1]){
            return data[i];
        }
    }

    return data[data.length-1];
}
```

APPENDIX B:

```
private static final HashMap<Integer, Integer> map = new HashMap<>();
public static int fibonacci(int n){
    System.out.println("Calculating fibonacci #" + n);
    if(n < 0){
        throw new IllegalArgumentException("Fibonacci numbers must be >= 0");
    }

    if(n <= 1){
        return n;
    }

    if(map.containsKey(n)){
        return map.get(n);
    }

    int result = fibonacci( n: n-2) + fibonacci( n: n-1);
    map.put(n, result);

    return result;
}
```

heme